Separate Chaining Collision Handling Technique in Hashing

Last Updated : 04 Mar, 2025

Separate Chaining is a collision handling technique. Separate chaining is one of the most popular and commonly used techniques in order to handle collisions. In this article, we will discuss about what is Separate Chain collision handling technique, its advantages, disadvantages, etc.

There are mainly two methods to handle collision:

- Separate Chaining
- Open Addressing

In this article, only separate chaining is discussed. We will be discussing Open addressing in the next post

Separate Chaining:

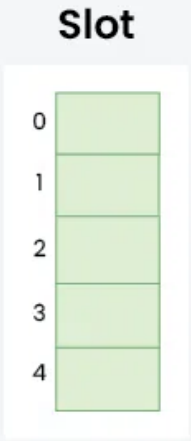
The idea behind separate chaining is to implement the array as a linked list called a chain.

> ***Linked List (or a Dynamic Sized Array)*** *is used to implement this technique. So what happens is, when multiple elements are hashed into the same slot index, then these elements are inserted into a singly-linked list which is known as a chain.*

Here, all those elements that hash into the same slot index are inserted into a linked list. Now, we can use a key K to search in the linked list by just linearly traversing. If the intrinsic key for any entry is equal to K then it means that we have found our entry. If we have reached the end of the linked list and yet we haven't found our entry then it means that the entry does not exist. Hence, the conclusion is that in separate chaining, if two different elements have the same hash value then we store both the elements in the same linked list one after the other.

Example: Let us consider a simple hash function as "**key mod 5**" and a sequence of keys as 12, 22, 15, 25

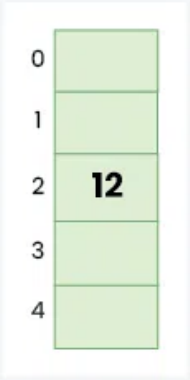
Separate Chaining



Step 01

Empty hash table with range of hash values from **0 to 4** according to the hash function provided.

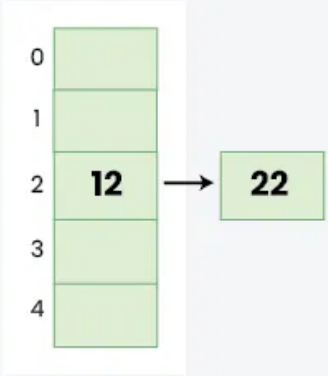
Separate Chaining



Step 02

The first key to be inserted is **12** which is mapped to **slot 2 (12%5=2)**.

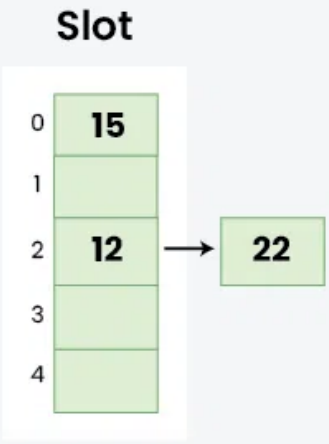
Separate Chaining



Step 03

The next key is **22** which is mapped to **slot 2 (22%5=2)** but **slot 2** is already occupied by **key 12**. Separate chaining will handle collision by creating a linked list to **slot 2.**

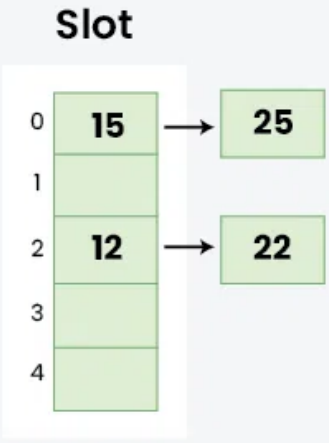
Separate Chaining



Step 04

The next key is **15** which is mapped to **slot 0 (15%5=0).**

Separate Chaining



Step 05

The next key is **25** which is mapped to **slot 0 (25%5=0)**. But slot 0 is already occupied by **key 25**. Again, Separate chaining will handle collision by creating a linked list to **slot 2**.

Implementation

Please refer Program for hashing with chaining for implementation.

Advantages:

- Simple to implement.
- Hash table never fills up, we can always add more elements to the chain.
- Less sensitive to the hash function or load factors.
- It is mostly used when it is unknown how many and how frequently keys may be inserted or deleted.

Disadvantages:

- The cache performance of chaining is not good as keys are stored using a linked list. Open addressing provides better cache performance as everything is stored in the same table.
- Wastage of Space (Some Parts of the hash table are never used)
- If the chain becomes long, then search time can become O(n) in the worst case
- Uses extra space for links

Performance of Chaining:

Performance of hashing can be evaluated under the assumption that each key is equally likely to be hashed to any slot of the table (simple uniform hashing).

m = Number of slots in hash table
n = Number of keys to be inserted in hash table

Load factor α = n/m
Expected time to search = O(1 + α)
Expected time to delete = O(1 + α)

Time to insert = O(1)
Time complexity of search insert and delete is O(1) if α is O(1)

Data Structures For Storing Chains:

Below are different options to create chains.

- The advantage of linked list implementation is insert is O(1) in the worst case.
- The advantage of array is cache friendliness, but the insert operation can be O(1) in cases when we have to resize the array.
- The advantage of Self Balancing BST is the worst case is bounded by O(Log (len)) for all operations

1. Linked lists

- Search: O(len) where len = length of chain
- Delete: O(len)
- Insert: O(1)
- Not cache friendly

2. Dynamic Sized Arrays (Vectors in C++, ArrayList in Java, list in Python)

- Search: O(len) where len = length of chain
- Delete: O(len)
- Insert: O(l)
- Cache friendly

3. Self Balancing BST (AVL Trees, Red-Black Trees)

- Search: O(log(len)) where len = length of chain
- Delete: O(log(len))
- Insert: O(log(len))
- Not cache friendly

- Java 8 onward versions use this for HashMap

Related Posts:

Open Addressing for Collision Handling

Hashing | Set 1 (Introduction)

Separate Chaining

Next Article

Open Addressing Collision Handling technique in Hashing

Similar Reads

Hashing in Data Structure

Hashing is a technique used in data structures that efficiently stores and retrieves data in a way that allows for quick access. Hashing involves mapping data to a specific index in a hash table (an array of...

3 min read

Introduction to Hashing

Hashing refers to the process of generating a small sized output (that can be used as index in a table) from an input of typically large and variable size. Hashing uses mathematical formulas known as hash...

7 min read

What is Hashing?

Hashing refers to the process of generating a fixed-size output from an input of variable size using the mathematical formulas known as hash functions. This technique determines an index or location for the...

3 min read

Index Mapping (or Trivial Hashing) with negatives allowed

Index Mapping (also known as Trivial Hashing) is a simple form of hashing where the data is directly mapped to an index in a hash table. The hash function used in this method is typically the identity...

7 min read

Separate Chaining Collision Handling Technique in Hashing

Separate Chaining is a collision handling technique. Separate chaining is one of the most popular and commonly used techniques in order to handle collisions. In this article, we will discuss about what is...

3 min read

Open Addressing Collision Handling technique in Hashing

Open Addressing is a method for handling collisions. In Open Addressing, all elements are stored in the hash table itself. So at any point, the size of the table must be greater than or equal to the total number...

7 min read

Double Hashing

Double hashing is a collision resolution technique used in hash tables. It works by using two hash functions to compute two different hash values for a given key. The first hash function is used to comput...

15+ min read

Load Factor and Rehashing

Prerequisites: Hashing Introduction and Collision handling by separate chaining How hashing works: For insertion of a key(K) - value(V) pair into a hash map, 2 steps are required: K is converted into a small...

15+ min read

Easy problems on Hashing

Intermediate problems on Hashing